

Introduction to While Loops

Christopher Martin - *Bowdoin College* 
c.martin@bowdoin.edu 

Last week, we saw how **for** loops can allow us to:

- Repeat instructions a set number of times
- Repeat instructions once for each piece of data in a collection

This week, we will focus on **while** loops and different behaviors we can accomplish now using nested loops



While Loops

While loops lend themselves to repeating a list of instructions an unknown number of times until a specified condition is met
This means we know when to stop but not how many repetitions it will take...

Their syntax is as follows:

```
1 while CONDITION:
2     Code we want to repeat 'while' the condition is True
3     ...
4     UPDATE
```

- **CONDITION** can be any boolean expression, just like an if statement!

The code inside the while loop will continue to execute until the condition evaluates to `False`

- **UPDATE** code to potentially change the condition's result

Without changing the state of its condition, a while loop can run infinitely!

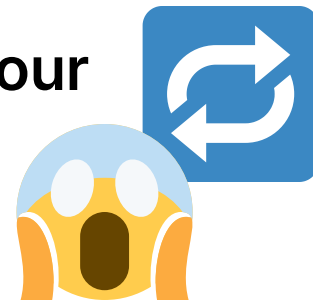
While Loops - Infinite Repetition

```
1 import random
2
3 random.seed(42) # 'Seed' the random number generator so we get consistent random results
4
5 target = 3
6 number = random.randint(1, 10) # Generate a random number between 1 and 10
7 steps = 1
8
9 while number != target:          # Repeat 'while' we haven't generated our target number
10     print(number, end=" ")      # Print the current number
11     number = random.randint(1, 10) # Generate a new random number between 1 and 10
12     steps = steps + 1           # Add 1 to the number of steps we have taken
13
14 print(target)
15 print(f"Randomly generated a {target} after {steps} guess(es)")
```

```
2 1 5 4 4 3
Randomly generated a 3 after 6 guess(es)
```

Keyboard Interrupt

This will be the first time we can accidentally cause our code to run forever if we aren't careful!



If that happens, what can we do?

You can forcibly end your program by pressing `ctrl + c` at the same time!

You will see the following in your terminal if it worked correctly:

```
1 raise KeyboardInterrupt
```

```
Traceback (most recent call last):  
  File "/app/output.s", line 1, in <module>  
    raise KeyboardInterrupt  
KeyboardInterrupt
```

While vs For

While

Can do anything a for loop can!

Maybe just written out a little differently!

```
1 my_list: list[int] = [1, 2, 3]
2
3 pos = 0
4 while pos < len(my_list):
5     print(my_list[pos])
6     pos = pos + 1
```

```
1
2
3
```

For

Can do anything a while loop can!

Maybe just written out a little differently!

```
1 my_list: list[int] = [1, 2, 3]
2
3 for num in my_list:
4     print(num)
```

```
1
2
3
```

User Input

We use while loops a lot when dealing with users and their input...

Why? Because we have no idea what the user is going to try to do!



We won't know in advance what they want to do, what they will enter, or how many tries it will take them to do what we want!

As a result, we tend to use while loops more than for loops for user input because of this uncertainty, it also gives us a chance to force valid inputs!

User Input

Let's look at an example:

```
1 str_number = input("Please enter an integer!")
2 invalid = True # Assume the input is bad to validate it
3
4 while invalid: # Loop 'while' the input is invalid
5     if str_number[0] in "+-" and str_number[1:].isdigit():
6         invalid = False # The input was a valid +/- int!
7     elif str_number.isdigit():
8         invalid = False # The input was valid int!
9     # Potentially more checks here ...
10    else:
11        # The input was invalid! Ask again and keep invalid True!
12        str_number = input("Please enter a positive number!")
13
14 number = int(str_number) # Safely convert once we know it is valid!
```